School Of Future Tech

Case Study Report

On

Stock Market Simulator

By

Krishiv Amitsinh Solanki

150096725072

1. Introduction to the Case Study

1.1 Role of Simulations

Simulation software bridges the gap between theoretical knowledge and practical application. By creating a risk-free environment, a Stock Market Simulator allows students and beginners to experiment with trading strategies, understand market volatility, and experience the emotional discipline required for trading without the fear of losing real capital.

1.2 Overview of the System

This case study focuses on the development of a console-based Stock Market Simulator built using the C++ programming language. The system mimics a real trading environment where stock prices fluctuate dynamically. It provides users with a virtual fund (₹50,000), allowing them to perform operations such as viewing live prices, buying stocks, selling holdings, and tracking their portfolio's performance in real-time.

2. Case Study Background

2.1 Background

Investors often suffer significant losses in the stock market due to a lack of understanding of price movements and portfolio management. Traditional learning methods involving textbooks do not offer the dynamic feedback loop required to master trading. There is a need for a lightweight, accessible tool that demonstrates how buy/sell decisions impact capital over time.

2.2 Abstract

The **Stock Market Simulator** is a C++ application designed to replicate the basic functionalities of a stock exchange. The problem statement requires the development of a system where stock prices are not static; they must change dynamically to simulate market volatility.

Key Features:

- **Dynamic Pricing:** Implementation of algorithms to randomize stock price fluctuations within a controlled range (-5% to +5%).
- **Virtual Currency System:** Users start with a fixed balance and must manage their funds to maximize profit.
- **Portfolio Management:** The system must track the quantity and average buy price of multiple stocks simultaneously.
- **Profit/Loss Calculation:** Real-time calculation of gains or losses based on current market prices versus investment cost.

3. Case Study Design

3.1 System Architecture

The system follows a modular architecture, separating data management from user interface logic. The application is event-driven, responding to user inputs from a main menu loop.

3.2 Data Structures

The system utilizes two primary structures to manage data:

Structure	Members	Description
struct Stock	string name double price	Represents a tradable asset in the market. Stores the current market price which updates dynamically.
struct Holding	string name int quantity double avgBuyPrice	Represents a user's ownership of a stock. Tracks how many shares are owned and the weighted average cost.

3.3 Core Modules

The program flow is controlled by the following functions:

- **updatePrices():** The engine of the simulator. It iterates through the stock array and applies a random percentage change to simulate market volatility.
- **buyStock():** Handles the validation of funds, deduction of balance, and updating the portfolio (including averaging the buy price for existing holdings).
- **sellStock():** Calculats revenue, updates cash balance, realizes profit/loss, and removes or reduces holdings.
- **viewPortfolio():** A reporting module that displays current investments, compares current price vs. buy price, and calculates net worth.

4. Methods & Algorithms Technology Applied

4.1 Stock Price Simulation Algorithm

To simulate market volatility, the system uses a randomization algorithm. Unlike linear price changes, stock markets fluctuate. The algorithm generates a random percentage between -5.0% and +5.0% during every turn.

4.2 Portfolio Management (Weighted Average)

When a user buys more units of a stock they already own, the buy price must be averaged. The **Weighted Average Price** method is applied:

NewAvgPrice = ((OldQty * OldAvg) + (NewQty * CurrentPrice)) / (OldQty + NewQty)

4.3 Technology Stack

- **Language:** C++
- **Libraries:**
  - <iostream>: Standard I/O operations.
  - <iomanip>: For formatting currency output (precision, width).
  - <cstdlib> & <ctime>: For random number generation functionality.
  - <string>: For handling stock names.

5. Implementation Details and Snapshots

5.1 Key Code Sections

Global Configuration

The system defines constants for limits and initializes the available stocks.

Buying Logic

The buyStock function ensures the user cannot spend more than their available balance. It also handles the logic for adding to an existing holding vs. creating a new one.

5.2 Sample Output Snapshots

1. Main Menu and Stock View

The user is presented with available stocks and their current prices. Note the formatted output using set w.

2. Purchasing Stocks

When a purchase is made, the cost is calculated, deducted from the balance, and a confirmation is shown.

3. Portfolio Summary

The portfolio view calculates the unrealized Profit/Loss (P/L) by comparing the Average Buy Price with the Current Market Price.

6. Results and Conclusion

6.1 System Effectiveness

The Stock Market Simulator effectively meets the requirements of the case study. It successfully creates a closed-loop financial system where:

- Users can execute trades based on real-time decision making.
- The system prevents invalid operations (e.g., buying with insufficient funds, selling unowned stocks).
- Arithmetic precision is maintained for currency calculations.
- The random price movement creates an engaging and unpredictable environment.

6.2 Learning Outcomes

Through the implementation of this project, the following learning outcomes were achieved:

- **Logic Building:** Implementation of complex conditions for trading and portfolio updates.
- **Structs and Arrays:** Practical application of managing collections of custom data types.
- **Algorithmic Thinking:** Designing the logic for weighted averages and random volatility.
- **Financial Basics:** Understanding the mechanics of capital markets, cost basis, and profit realization.

6.3 Future Enhancements

The current system is console-based. Future iterations could include:

- Integration with a live stock market API for real-world data.
- Graphical User Interface (GUI) for better visualization of charts.
- Historical data tracking to show price trends over time.
- Transaction history logs (Ledger).

6.4 Conclusion

This case study demonstrates that C++ is a powerful tool for financial modeling. The project provides a solid foundation for understanding how trading platforms operate on a logical level, blending computer science concepts with financial literacy education.

7. References

- **C++ Documentation:** cplusplus.com - Reference for STL vectors, I/O manipulation, and standard libraries.
- **Financial Concepts:** Investopedia - Definitions of weighted average cost and portfolio management.